Inventory Management System

Using MERN Stack

Step 2: Database Implementation

Database Creation, Queries,
Execution and Testing

Web Engineering Lab Project

Jashore University of Science and Technology

Contents

1 Introduction **3**

2 Database Creation **3**

2.1 Platform . . . 3
2.2 Collections . . . 3
2.3 How the collections are created . . . 3
2.4 Practice database . . . 4

3 How to Execute the Queries **5**

4 Create Queries (Insert) **5**

C1. Insert one category . . . 5
C2. Insert several products at once . . . 6
C3. Try to insert a duplicate SKU (must fail) . . . 6

5 Read Queries (Find) **6**

R1. List all active products . . . 6
R2. Find one product by its SKU . . . 7
R3. Products in a price range . . . 7
R4. Search products by name or SKU . . . 8
R5. Low stock and out of stock products . . . 8
R6. Products of selected categories . . . 8
R7. Count the completed sales . . . 9
R8. All sales of one customer . . . 9
R9. Completed sales that still have a due amount . . . 9
R10. Sales in a date range . . . 10
R11. Search inside the items of a sale . . . 10
R12. Pagination (page 2, three products per page) . . . 11

6 Update Queries **11**

U1. Change the price of a product . . . 11
U2. Add stock when products arrive . . . 11
U3. Sell more than the stock (must fail) . . . 12
U4. Sell an available quantity (must succeed) . . . 12
U5. Update many documents at once . . . 12
U6. Record a customer payment . . . 13

7 Delete Queries **13**

D1. Soft delete (what the application does) . . . 13
D2. Delete one document . . . 14
D3. Delete many documents . . . 14

8 Aggregation Queries (Reports) 14

A1. Number of products and total stock per category 14

A2. Total value of the stock 15

A3. Sales summary 15

A4. Due amount per customer 15

A5. Best selling products 16

A6. Gross profit 16

A7. Sales per day (Bangladesh time) 17

A8. Money owed to each supplier 17

A9. Money received by payment method 18

9 Index Queries 18

I1. Show the indexes of a collection 18

I2. Check that a query uses the index 18

10 How the Application Uses These Queries 19

11 Test Checklist 19

1 Introduction

This document covers the second step of the project, **Database Implementation**. It has three parts:

1. **Create the database:** where the database is, which collections it has and how they are created.
2. **Write the queries:** 35 queries in six groups (Create, Read, Update, Delete, Aggregation and Index).
3. **Execute and test:** every query is shown with its expected result, and a checklist is given at the end.

The database is **MongoDB**, a document database. The comparison below helps readers who know SQL.

SQL (relational)	**MongoDB (document)**
Database	Database
Table	Collection
Row	Document (JSON-like object)
Column	Field
Primary key	`_id` (created automatically)
Foreign key	Reference: the `_id` of another document
JOIN	`$lookup` stage, or embedded documents
`SELECT ... WHERE`	`find({ filter })`
`GROUP BY`	`$group` stage of an aggregation

Table 1: SQL terms and the matching MongoDB terms

2 Database Creation

2.1 Platform

The database is hosted on **MongoDB Atlas**, the cloud service of MongoDB. The cluster runs in the AWS Mumbai region (`ap-south-1`) as a replica set of three nodes, so the data is stored in three copies. The database name is `inventory_management_system`.

The backend (Node.js with Express) connects to the database with the Mongoose library. The connection string is kept in the environment variable `MONGO_URI` on the server. It is never written in the source code.

2.2 Collections

The database has 13 collections. Their fields and relationships are shown in the database diagram of Step 1.

2.3 How the collections are created

MongoDB has no `CREATE TABLE` command. A collection is created automatically when the first document is inserted. The structure and the rules of each collection are written in the backend as a **Mongoose schema** (folder `server/src/models`). A part of the product schema is shown below.

Collection	What it stores
users	User accounts with a role (Administrator, Manager or Sales Staff)
categories	Product categories, for example Smartphone or Laptop
brands	Product brands, for example Samsung or Dell
products	Products with SKU, prices, current stock and minimum stock level
customers	Customers who can buy on credit
suppliers	Suppliers from whom products are bought
sales	Sales, with the sold lines embedded in the items array
payments	Money received from customers for a sale
purchases	Purchases from suppliers, with the lines embedded in items
supplierpayments	Money paid to suppliers for a purchase
returns	Products returned by customers or sent back to suppliers
stockmovements	History of every stock change (sale, purchase, return, adjustment)
auditlogs	Who did what and when

Table 2: Collections of the database

```
const productSchema = new mongoose.Schema(
  {
    name:          { type: String, required: true, trim: true, maxlength: 150 },
    sku:           { type: String, required: true, unique: true, uppercase: true },
    category:      { type: ObjectId, ref: "Category", required: true },
    brand:         { type: ObjectId, ref: "Brand", required: true },
    purchasePrice: { type: Number, min: 0, default: 0 },
    sellingPrice:  { type: Number, required: true, min: 0 },
    currentStock:  { type: Number, min: 0, default: 0 },
    minStockLevel: { type: Number, min: 0, default: 0 },
    isActive:      { type: Boolean, default: true },
  },
  { timestamps: true }
);
```

The schema gives the same protection as constraints in SQL:

- required: true works like NOT NULL.
- unique: true creates a unique index, like a UNIQUE constraint.
- min: 0 stops negative prices and negative stock, like a CHECK constraint.
- ref: "Category" marks a reference to another collection, like a foreign key.
- timestamps: true adds createdAt and updatedAt to every document.

2.4 Practice database

Create, Update and Delete queries change data. To keep the real data safe, the queries of this document are executed in a separate database named stockpilot_practice on the same cluster. The script seed-practice.js fills it with a small set of sample data: 8 products, 4 categories, 4 brands, 3 customers, 2 suppliers, 6 sales and 4 purchases. The collection names and field names are the same as in the real database, so every query also works there.

3 How to Execute the Queries

The queries are written for **mongosh**, the MongoDB shell. The shell is built into MongoDB Compass.

1. Open MongoDB Compass and connect to the Atlas cluster with the connection string.
2. Open the **mongosh** panel at the bottom of the window.
3. Type `use stockpilot_practice` and press Enter.
4. Paste the whole file `seed-practice.js` and press Enter. The message *Practice data is ready: 8 products, 6 sales, 4 purchases.* appears.
5. Open `queries.js`. Copy one block at a time, paste it and press Enter. Compare the output with the expected result in this document.

Three points to remember:

- **Run the queries in order.** Later results depend on earlier changes (for example, A9 includes the payment of U6). To start again, run `seed-practice.js` again.
- When a block has several statements, this document shows the result of the **last** statement.
- An `_id` that MongoDB generates is different on every run, so it is shown here with three dots in place of the value. The shell may also break long lines at other places.

The Read, Aggregation and Index queries do not change anything. They can also be run on the real database (`use inventory_management_system`); the results will then show the real data.

4 Create Queries (Insert)

These queries add new documents. MongoDB creates a collection automatically when the first document is inserted, and it gives every document a unique `_id`.

C1. Insert one category

`insertOne` adds a single document. The result shows the `_id` that MongoDB generated (it will be different on every run).

```
db.categories.insertOne({
  name: "Headphone",
  description: "Wired and wireless headphones",
  isActive: true
})
```

Expected result:

```
{ acknowledged: true, insertedId: ObjectId('...') }
```

C2. Insert several products at once

A product stores the `_id` of its category and brand (a reference). So first the category and the brand are found by name, and then `insertMany` adds two products in one call.

```
var headphone = db.categories.findOne({ name: "Headphone" });
var jbl = db.brands.findOne({ name: "JBL" });

db.products.insertMany([
  { name: "JBL Tune 520BT", sku: "JBL-T520-001",
    category: headphone._id, brand: jbl._id,
    purchasePrice: 4200, sellingPrice: 5200, currentStock: 8, minStockLevel: 3,
    unit: "pcs", isActive: true },
  { name: "JBL Tune 110", sku: "JBL-T110-001",
    category: headphone._id, brand: jbl._id,
    purchasePrice: 700, sellingPrice: 950, currentStock: 25, minStockLevel: 10,
    unit: "pcs", isActive: true }
])
```

Expected result:

```
{ acknowledged: true, insertedIds: { '0': ObjectId('...'), '1': ObjectId('...') } }
```

C3. Try to insert a duplicate SKU (must fail)

The `sku` field has a unique index. This insert uses a SKU that already exists, so MongoDB rejects it with error `E11000`. This test proves that two products can never share one SKU.

```
db.products.insertOne({
  name: "Fake Galaxy",
  sku: "SAM-S24-001",
  sellingPrice: 1000
})
```

Expected error:

```
MongoServerError: E11000 duplicate key error collection: stockpilot_practice.products index:
    sku_1 dup key: { sku: "SAM-S24-001" }
```

5 Read Queries (Find)

These queries only read data, so they are safe to run on any database. The first argument of `find` is the filter (which documents), and the second is the projection (which fields to show).

R1. List all active products

Filter: `isActive` is true. Projection: show only four fields and hide `_id`. The result is sorted by name (1 means ascending).

```
db.products.find(
  { isActive: true },
  { _id: 0, name: 1, sku: 1, sellingPrice: 1, currentStock: 1 }
).sort({ name: 1 })
```

Expected result:

```
[
  { name: 'Dell Inspiron 15', sku: 'DEL-INS15-001', sellingPrice: 70000, currentStock: 6 },
  {
    name: 'Dell Latitude 5440',
    sku: 'DEL-LAT5440-001',
    sellingPrice: 110000,
    currentStock: 0
  },
  { name: 'Galaxy A15', sku: 'SAM-A15-001', sellingPrice: 21000, currentStock: 12 },
  { name: 'Galaxy S24', sku: 'SAM-S24-001', sellingPrice: 95000, currentStock: 3 },
  { name: 'JBL Flip 6', sku: 'JBL-FLP6-001', sellingPrice: 12500, currentStock: 4 },
  { name: 'JBL Go 3', sku: 'JBL-GO3-001', sellingPrice: 4000, currentStock: 15 },
  { name: 'JBL Tune 110', sku: 'JBL-T110-001', sellingPrice: 950, currentStock: 25 },
  { name: 'JBL Tune 520BT', sku: 'JBL-T520-001', sellingPrice: 5200, currentStock: 8 },
  { name: 'M331 Silent Mouse', sku: 'LOG-M331-001', sellingPrice: 1800, currentStock: 30 }
]
```

R2. Find one product by its SKU

`findOne` returns a single document (or `null` when nothing matches). Because SKU is unique, this is the fastest way to open one product.

```
db.products.findOne({ sku: "JBL-GO3-001" })
```

Expected result:

```
{
  _id: ObjectId('650000000000000000000305'),
  name: 'JBL Go 3',
  sku: 'JBL-GO3-001',
  category: ObjectId('650000000000000000000103'),
  brand: ObjectId('650000000000000000000203'),
  purchasePrice: 3200,
  sellingPrice: 4000,
  currentStock: 15,
  minStockLevel: 5,
  unit: 'pcs',
  isActive: true
}
```

R3. Products in a price range

`$gte` means greater than or equal, `$lte` means less than or equal. The result is sorted from the highest price to the lowest (-1 means descending).

```
db.products.find(
  { sellingPrice: { $gte: 10000, $lte: 80000 } },
  { _id: 0, name: 1, sellingPrice: 1 }
).sort({ sellingPrice: -1 })
```

Expected result:

```
[
  { name: 'Dell Inspiron 15', sellingPrice: 70000 },
  { name: 'Galaxy A15', sellingPrice: 21000 },
  { name: 'JBL Flip 6', sellingPrice: 12500 }
]
```

R4. Search products by name or SKU

`$regex` matches a part of the text and option `i` ignores upper and lower case. `$or` means that a match in the name or in the SKU is enough. The search box of the Products page works in the same way.

```
db.products.find(
  { $or: [
      { name: { $regex: "galaxy", $options: "i" } },
      { sku:  { $regex: "galaxy", $options: "i" } }
  ] },
  { _id: 0, name: 1, sku: 1 }
)
```

Expected result:

```
[ { name: 'Galaxy S24', sku: 'SAM-S24-001' }, { name: 'Galaxy A15', sku: 'SAM-A15-001' } ]
```

R5. Low stock and out of stock products

Here two fields of the same document are compared, so `$expr` is needed. A product is listed when its `currentStock` is less than or equal to its `minStockLevel`.

```
db.products.find(
  { isActive: true, $expr: { $lte: ["$currentStock", "$minStockLevel"] } },
  { _id: 0, name: 1, currentStock: 1, minStockLevel: 1 }
)
```

Expected result:

```
[
  { name: 'Galaxy S24', currentStock: 3, minStockLevel: 5 },
  { name: 'Dell Latitude 5440', currentStock: 0, minStockLevel: 2 },
  { name: 'JBL Flip 6', currentStock: 4, minStockLevel: 4 }
]
```

R6. Products of selected categories

`$in` matches any value of a list. This query shows every product that is a Smartphone or a Laptop.

```
var ids = db.categories.find({ name: { $in: ["Smartphone", "Laptop"] } })
  .toArray().map(function (c) { return c._id; });

db.products.find({ category: { $in: ids } }, { _id: 0, name: 1, sku: 1 })
```

Expected result:

```
[
  { name: 'Galaxy S24', sku: 'SAM-S24-001' },
  { name: 'Galaxy A15', sku: 'SAM-A15-001' },
  { name: 'Dell Inspiron 15', sku: 'DEL-INS15-001' },
  { name: 'Dell Latitude 5440', sku: 'DEL-LAT5440-001' }
]
```

R7. Count the completed sales

`countDocuments` returns only a number. There are 6 sales in total, but one is a draft and one is cancelled.

```
db.sales.countDocuments({ status: "COMPLETED" })
```

Expected result:

```
4
```

R8. All sales of one customer

The customer is found by name first. Then the sales that hold this customer's `_id` are listed, newest first.

```
var rahim = db.customers.findOne({ name: "Rahim Traders" });

db.sales.find(
  { customer: rahim._id },
  { _id: 0, saleNumber: 1, grandTotal: 1, status: 1, paymentStatus: 1 }
).sort({ saleDate: -1 })
```

Expected result:

```
[
  { saleNumber: 'SAL-000005', grandTotal: 12000, status: 'DRAFT', paymentStatus: 'UNPAID' },
  {
    saleNumber: 'SAL-000001',
    grandTotal: 98000,
    status: 'COMPLETED',
    paymentStatus: 'PARTIALLY_PAID'
  }
]
```

R9. Completed sales that still have a due amount

`$gt` means greater than. Draft and cancelled sales are left out, because only a completed sale creates a real due.

```
db.sales.find(
  { status: "COMPLETED", outstandingAmount: { $gt: 0 } },
  { _id: 0, saleNumber: 1, grandTotal: 1, paidAmount: 1, outstandingAmount: 1 }
)
```

Expected result:

```
[
  { saleNumber: 'SAL-000001', grandTotal: 98000, paidAmount: 50000, outstandingAmount: 48000
    },
  { saleNumber: 'SAL-000003', grandTotal: 82500, paidAmount: 0, outstandingAmount: 82500 }
]
```

R10. Sales in a date range

Dates are compared with `$gte` (from) and `$lt` (before). This query shows the completed sales from 1 November to 5 November 2026.

```
db.sales.find(
  { status: "COMPLETED",
    saleDate: { $gte: ISODate("2026-11-01T00:00:00Z"),
                $lt:  ISODate("2026-11-06T00:00:00Z") } },
  { _id: 0, saleNumber: 1, saleDate: 1, grandTotal: 1 }
)
```

Expected result:

```
[
  {
    saleNumber: 'SAL-000001',
    saleDate: ISODate('2026-11-02T05:00:00.000Z'),
    grandTotal: 98000
  },
  {
    saleNumber: 'SAL-000002',
    saleDate: ISODate('2026-11-03T08:30:00.000Z'),
    grandTotal: 8000
  },
  {
    saleNumber: 'SAL-000003',
    saleDate: ISODate('2026-11-05T10:00:00.000Z'),
    grandTotal: 82500
  }
]
```

R11. Search inside the items of a sale

The products of a sale are stored inside the sale document, in the `items` array. `$elemMatch` finds the sales where one single line is JBL Go 3 with a quantity of 2 or more.

```
db.sales.find(
  { items: { $elemMatch: { sku: "JBL-GO3-001", quantity: { $gte: 2 } } } },
  { _id: 0, saleNumber: 1, status: 1, "items.productName": 1, "items.quantity": 1 }
)
```

Expected result:

```
[
  {
    saleNumber: 'SAL-000002',
    status: 'COMPLETED',
    items: [ { productName: 'JBL Go 3', quantity: 2 } ]
  },
  {
    saleNumber: 'SAL-000005',
    status: 'DRAFT',
    items: [ { productName: 'JBL Go 3', quantity: 3 } ]
  }
]
```

R12. Pagination (page 2, three products per page)

`skip` jumps over the first rows and `limit` sets the page size. For page number p the rule is skip = (p - 1) × limit. The Products page of the website uses the same rule with 25 products per page.

```
db.products.find({}, { _id: 0, name: 1 })
  .sort({ name: 1 })
  .skip(3)
  .limit(3)
```

Expected result:

```
[ { name: 'Galaxy S24' }, { name: 'JBL Flip 6' }, { name: 'JBL Go 3' } ]
```

6 Update Queries

An update has two parts: a filter (which document) and the change. In the result, `matchedCount` is the number of documents found and `modifiedCount` is the number really changed.

U1. Change the price of a product

`$set` changes only the named field. All other fields of the product stay the same.

```
db.products.updateOne(
  { sku: "JBL-GO3-001" },
  { $set: { sellingPrice: 4200 } }
)
```

Expected result:

```
{ acknowledged: true, insertedId: null, matchedCount: 1, modifiedCount: 1, upsertedCount: 0 }
```

U2. Add stock when products arrive

`$inc` adds a number to the current value inside the database. It is safer than reading the stock, adding in the program and writing it back, because two users can work at the same time. The second statement shows the new stock (4 + 10).

```
db.products.updateOne(
  { sku: "JBL-FLP6-001" },
  { $inc: { currentStock: 10 } }
);

db.products.findOne({ sku: "JBL-FLP6-001" }, { _id: 0, name: 1, currentStock: 1 })
```

Expected result:

```
{ name: 'JBL Flip 6', currentStock: 14 }
```

U3. Sell more than the stock (must fail)

Galaxy S24 has only 3 in stock. The filter says: update only if `currentStock` is 5 or more. No document matches, so nothing changes (`matchedCount` is 0) and the stock can never go below zero. The real project completes a sale with this same technique.

```
db.products.updateOne(
  { sku: "SAM-S24-001", currentStock: { $gte: 5 } },
  { $inc: { currentStock: -5 } }
)
```

Expected result:

```
{ acknowledged: true, insertedId: null, matchedCount: 0, modifiedCount: 0, upsertedCount: 0 }
```

U4. Sell an available quantity (must succeed)

The same query with quantity 2. Now the filter matches, so the stock goes down from 3 to 1. The check and the change happen in one step.

```
db.products.updateOne(
  { sku: "SAM-S24-001", currentStock: { $gte: 2 } },
  { $inc: { currentStock: -2 } }
);

db.products.findOne({ sku: "SAM-S24-001" }, { _id: 0, name: 1, currentStock: 1 })
```

Expected result:

```
{ name: 'Galaxy S24', currentStock: 1 }
```

U5. Update many documents at once

`updateMany` changes every document that matches. Here the minimum stock level of all Logitech products is set to 15.

```
var logitech = db.brands.findOne({ name: "Logitech" });

db.products.updateMany(
  { brand: logitech._id },
  { $set: { minStockLevel: 15 } }
)
```

Expected result:

```
{ acknowledged: true, insertedId: null, matchedCount: 2, modifiedCount: 2, upsertedCount: 0 }
```

U6. Record a customer payment

Karim Electronics pays 30,000 for sale SAL-000003 (due 82,500). One document is added to `payments`, and the sale is updated: paid goes up, due goes down, and the status changes. In the real project these two steps run inside one transaction, so both succeed or both fail.

```
var sale = db.sales.findOne({ saleNumber: "SAL-000003" });

db.payments.insertOne({
  sale: sale._id, customer: sale.customer,
  amount: 30000, method: "CASH", reference: ""
});

db.sales.updateOne(
  { _id: sale._id, outstandingAmount: { $gte: 30000 } },
  { $inc: { paidAmount: 30000, outstandingAmount: -30000 },
    $set: { paymentStatus: "PARTIALLY_PAID" } }
);

db.sales.findOne(
  { saleNumber: "SAL-000003" },
  { _id: 0, saleNumber: 1, paidAmount: 1, outstandingAmount: 1, paymentStatus: 1 }
)
```

Expected result:

```
{
  saleNumber: 'SAL-000003',
  paidAmount: 30000,
  outstandingAmount: 52500,
  paymentStatus: 'PARTIALLY_PAID'
}
```

7 Delete Queries

The real project never removes a product, customer or supplier, because old sales and purchases still point to them. It only switches them off (`isActive: false`). Real delete queries are shown here on the practice data only.

D1. Soft delete (what the application does)

The product is not removed. It is only marked as inactive, so it disappears from the sale form but old records stay correct.

```
db.products.updateOne(
  { sku: "JBL-T110-001" },
  { $set: { isActive: false } }
)
```

Expected result:

```
{ acknowledged: true, insertedId: null, matchedCount: 1, modifiedCount: 1, upsertedCount: 0 }
```

D2. Delete one document

`deleteOne` removes the first document that matches the filter.

```
db.products.deleteOne({ sku: "JBL-T110-001" })
```

Expected result:

```
{ acknowledged: true, deletedCount: 1 }
```

D3. Delete many documents

`deleteMany` removes every match. Here all inactive products with zero stock are removed (only Old Test Mouse matches). Always run the same filter with `find` first, to see what will be deleted.

```
db.products.deleteMany({ isActive: false, currentStock: 0 })
```

Expected result:

```
{ acknowledged: true, deletedCount: 1 }
```

8 Aggregation Queries (Reports)

An aggregation is a pipeline: the documents pass through several stages, one after another. The most used stages are `$match` (filter), `$group` (calculate totals), `$lookup` (join another collection), `$unwind` (open an array), `$project` (choose fields) and `$sort`. The reports of the website are built this way.

A1. Number of products and total stock per category

`$lookup` joins each product with its category (like a JOIN in SQL). `$unwind` turns the joined array into one object. `$group` then counts the products and adds the stock for each category name.

```
db.products.aggregate([
  { $match: { isActive: true } },
  { $lookup: { from: "categories", localField: "category",
               foreignField: "_id", as: "category" } },
  { $unwind: "$category" },
  { $group: { _id: "$category.name",
              products: { $sum: 1 },
              totalStock: { $sum: "$currentStock" } } },
  { $sort: { _id: 1 } }
])
```

Expected result:

```
[
  { _id: 'Headphone', products: 1, totalStock: 8 },
  { _id: 'Laptop', products: 2, totalStock: 6 },
  { _id: 'Mouse', products: 1, totalStock: 30 },
  { _id: 'Smartphone', products: 2, totalStock: 13 },
  { _id: 'Speaker', products: 2, totalStock: 29 }
]
```

A2. Total value of the stock

For every active product, stock is multiplied by purchase price, and all results are added. `_id: null` means one single group for the whole collection.

```
db.products.aggregate([
  { $match: { isActive: true } },
  { $group: { _id: null,
              totalUnits: { $sum: "$currentStock" },
              stockValue: { $sum: {
                $multiply: ["$currentStock", "$purchasePrice"] } } } },
  { $project: { _id: 0 } }
])
```

Expected result:

```
[ { totalUnits: 86, stockValue: 943600 } ]
```

A3. Sales summary

Only completed sales are counted. This is the same calculation as the sales report of the website: number of sales, total sold, total received and total due.

```
db.sales.aggregate([
  { $match: { status: "COMPLETED" } },
  { $group: { _id: null,
              count: { $sum: 1 },
              totalSales: { $sum: "$grandTotal" },
              totalPaid: { $sum: "$paidAmount" },
              totalOutstanding: { $sum: "$outstandingAmount" } } },
  { $project: { _id: 0 } }
])
```

Expected result:

```
[ { count: 4, totalSales: 209500, totalPaid: 109000, totalOutstanding: 100500 } ]
```

A4. Due amount per customer

The unpaid completed sales are grouped by customer, and the customer name is added with `$lookup`. The customer with the highest due comes first.

```
db.sales.aggregate([
  { $match: { status: "COMPLETED", outstandingAmount: { $gt: 0 } } },
  { $group: { _id: "$customer",
              sales: { $sum: 1 },
              totalDue: { $sum: "$outstandingAmount" } } },
  { $lookup: { from: "customers", localField: "_id",
               foreignField: "_id", as: "customer" } },
  { $unwind: "$customer" },
  { $project: { _id: 0, customer: "$customer.name", sales: 1, totalDue: 1 } },
  { $sort: { totalDue: -1 } }
])
```

Expected result:

```
[
```

```
  { sales: 1, totalDue: 52500, customer: 'Karim Electronics' },
  { sales: 1, totalDue: 48000, customer: 'Rahim Traders' }
]
```

A5. Best selling products

`$unwind` makes one document for every line of the `items` array. Then the lines are grouped by product, and the quantity and the amount are added.

```
db.sales.aggregate([
  { $match: { status: "COMPLETED" } },
  { $unwind: "$items" },
  { $group: { _id: "$items.productName",
              quantitySold: { $sum: "$items.quantity" },
              revenue: { $sum: "$items.lineTotal" } } },
  { $sort: { quantitySold: -1, revenue: -1 } },
  { $limit: 3 }
])
```

Expected result:

```
[
  { _id: 'JBL Go 3', quantitySold: 2, revenue: 8000 },
  { _id: 'M331 Silent Mouse', quantitySold: 2, revenue: 3600 },
  { _id: 'Galaxy S24', quantitySold: 1, revenue: 95000 }
]
```

A6. Gross profit

Every sale line stores `costPrice`, the purchase price on the day of the sale. Profit of a line = (unit price - cost price) × quantity. Discounts are not subtracted here.

```
db.sales.aggregate([
  { $match: { status: "COMPLETED" } },
  { $unwind: "$items" },
  { $group: { _id: null,
              revenue: { $sum: "$items.lineTotal" },
              cost: { $sum: {
                $multiply: ["$items.costPrice", "$items.quantity"] } } } },
  { $project: { _id: 0, revenue: 1, cost: 1,
                grossProfit: { $subtract: ["$revenue", "$cost"] } } }
])
```

Expected result:

```
[ { revenue: 210100, cost: 184700, grossProfit: 25400 } ]
```

A7. Sales per day (Bangladesh time)

MongoDB stores dates in UTC. `$dateToString` with the timezone `Asia/Dhaka` changes each date to the local day first. Sale SAL-000004 was saved at 19:30 UTC on 7 November, so it is correctly counted on 8 November.

```
db.sales.aggregate([
  { $match: { status: "COMPLETED" } },
  { $group: { _id: { $dateToString: { format: "%Y-%m-%d", date: "$saleDate",
                                      timezone: "Asia/Dhaka" } },
              sales: { $sum: 1 },
              total: { $sum: "$grandTotal" } } },
  { $sort: { _id: 1 } }
])
```

Expected result:

```
[
  { _id: '2026-11-02', sales: 1, total: 98000 },
  { _id: '2026-11-03', sales: 1, total: 8000 },
  { _id: '2026-11-05', sales: 1, total: 82500 },
  { _id: '2026-11-08', sales: 1, total: 21000 }
]
```

A8. Money owed to each supplier

For a purchase: outstanding = grand total - returned amount - paid amount. `$subtract` calculates this for every completed purchase, and then it is added per supplier. Suppliers that are fully paid are left out.

```
db.purchases.aggregate([
  { $match: { status: "COMPLETED" } },
  { $group: { _id: "$supplier",
              purchases: { $sum: 1 },
              owed: { $sum: { $subtract: [
                { $subtract: ["$grandTotal", "$returnedAmount"] },
                "$paidAmount" ] } } } },
  { $match: { owed: { $gt: 0 } } },
  { $lookup: { from: "suppliers", localField: "_id",
               foreignField: "_id", as: "supplier" } },
  { $unwind: "$supplier" },
  { $project: { _id: 0, supplier: "$supplier.name", purchases: 1, owed: 1 } },
  { $sort: { owed: -1 } }
])
```

Expected result:

```
[
  { purchases: 2, owed: 285000, supplier: 'Tech Supply BD' },
  { purchases: 1, owed: 58500, supplier: 'Global Gadget Importers' }
]
```

A9. Money received by payment method

The `payments` collection is grouped by `method`. The cash total includes the payment of 30,000 from query U6.

```
db.payments.aggregate([
  { $group: { _id: "$method",
              payments: { $sum: 1 },
              total: { $sum: "$amount" } } },
  { $sort: { total: -1 } }
])
```

Expected result:

```
[
  { _id: 'MOBILE_BANKING', payments: 1, total: 50000 },
  { _id: 'CASH', payments: 2, total: 38000 },
  { _id: 'CARD', payments: 1, total: 21000 }
]
```

9 Index Queries

An index works like the index at the back of a book: MongoDB can jump to the right document without reading the whole collection. A unique index also stops duplicate values.

I1. Show the indexes of a collection

Every collection has an index on `_id`. The products collection also has the unique index on `sku` that made query C3 fail.

```
db.products.getIndexes()
```

Expected result:

```
[
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { sku: 1 }, name: 'sku_1', unique: true }
]
```

I2. Check that a query uses the index

`explain` shows how MongoDB runs a query. With the SKU index, MongoDB reads only one document to find one product (`totalDocsExamined: 1`), even if the collection has 10,000 products. The winning plan shows the stage `IXSCAN` (in MongoDB 8 it can be `EXPRESS_IXSCAN`). Without an index the stage would be `COLLSCAN`, which reads every document.

```
var stats = db.products.find({ sku: "JBL-GO3-001" })
  .explain("executionStats").executionStats;

({ nReturned: stats.nReturned,
   totalKeysExamined: stats.totalKeysExamined,
   totalDocsExamined: stats.totalDocsExamined })
```

Expected result:

```
{ nReturned: 1, totalKeysExamined: 1, totalDocsExamined: 1 }
```

10 How the Application Uses These Queries

The website does not send queries from the browser. The frontend calls the REST API, and the backend runs the query with Mongoose. The table shows where the techniques of this document are used in the project.

Query	Technique	Where it is used in the project
R4, R12	Search with a regular expression, `skip` and `limit`	Product list API (`productController.js`): search box and pages of 25 products
R5	`$expr` to compare two fields	Low stock filter of the Inventory page
U3, U4	Conditional update with `$inc`	Completing a sale (`saleController.js`): stock can never go below zero
U6	Insert and update together	Recording a payment, inside a transaction
D1	Soft delete with `isActive`	Deactivating products, customers and suppliers
A3, A6	`$match`, `$unwind`, `$group`	Sales report and profit report (`reportController.js`)
A8	Outstanding of a purchase	Supplier dues on the Payments page
C3, I1	Unique index	SKU of a product, sale number, purchase number

Table 3: Queries of this document and their use in the project

Transactions. Some actions change several collections. Completing a sale reduces the stock of each product, adds stock movement records and updates the sale. These changes run inside one MongoDB transaction (`session.withTransaction`). If any step fails, for example because one product has too little stock, all changes are cancelled and the database stays correct.

11 Test Checklist

Each query is executed in the practice database, and the output is compared with the expected result of this document. The box is ticked when both are the same.

ID	Query	Result matches
C1	Insert one category	☐
C2	Insert several products at once	☐
C3	Try to insert a duplicate SKU (must fail)	☐
R1	List all active products	☐
R2	Find one product by its SKU	☐
R3	Products in a price range	☐
R4	Search products by name or SKU	☐
R5	Low stock and out of stock products	☐
R6	Products of selected categories	☐
R7	Count the completed sales	☐
R8	All sales of one customer	☐
R9	Completed sales that still have a due amount	☐
R10	Sales in a date range	☐
R11	Search inside the items of a sale	☐
R12	Pagination (page 2, three products per page)	☐
U1	Change the price of a product	☐
U2	Add stock when products arrive	☐
U3	Sell more than the stock (must fail)	☐

ID	Query	Result matches
U4	Sell an available quantity (must succeed)	☐
U5	Update many documents at once	☐
U6	Record a customer payment	☐
D1	Soft delete (what the application does)	☐
D2	Delete one document	☐
D3	Delete many documents	☐
A1	Number of products and total stock per category	☐
A2	Total value of the stock	☐
A3	Sales summary	☐
A4	Due amount per customer	☐
A5	Best selling products	☐
A6	Gross profit	☐
A7	Sales per day (Bangladesh time)	☐
A8	Money owed to each supplier	☐
A9	Money received by payment method	☐
I1	Show the indexes of a collection	☐
I2	Check that a query uses the index	☐